

IF25-22017 PENGEMBANGAN APLIKASI MOBILE

Pertemuan 3

Dasar Compose

Multiplatform

Menyusun layar daftar laporan dengan composable, layout, modifier, dan LazyColumn

Muhammad Habib Algifari, S.Kom., M.T.I.

Tahun 2026



Program Studi Teknik Informatika
Fakultas Teknologi Industri
Institut Teknologi Sumatera

Tiga jawaban dari tiket keluar minggu lalu

Pertanyaan	Jawaban singkat
Kenapa status lebih aman sebagai sealed class?	Salah ketik ketahuan saat compile, dan when bisa memeriksa semua kemungkinan
Kenapa dua async 1000 ms dan 800 ms selesai sekitar 1000 ms?	Keduanya berjalan bersamaan, kita hanya menunggu yang paling lama
Apa yang terjadi pada Flow tanpa collect?	Tidak ada yang berjalan, karena Flow bersifat cold

Data sudah ada, layarnya belum

Minggu lalu kita punya data class Laporan, StatusLaporan, dan fungsi label(). Semuanya masih hidup di konsol. Hari ini data itu kita tampilkan sebagai layar daftar seperti di samping, dengan satu kode untuk Android dan desktop.

DISKUSI 2 MENIT

Lihat layar di samping. Bagian mana yang tersusun dari atas ke bawah, mana yang dari kiri ke kanan, dan mana yang bertumpuk?

LaporKampus

3 laporan aktif

Proyektor mati
GKU 1 R201

Baru

AC mati
GKU 2 R305

Diproses

Keran bocor
Perpustakaan

Selesai

Lapor

Saat pulang nanti, kalian bisa

1 **Berpikir deklaratif**
Menjelaskan bedanya mendeskripsikan UI dengan mengubahnya langkah demi langkah.

2 **Menyusun layar**
Memakai Column, Row, Box, dan Modifier dengan urutan yang tepat.

3 **Membuat komponen**
Menulis composable reusable, mengolah daftar dengan filter dan map, lalu menampilkannya.

Capaian ini mendukung **CPMK0501**: menerapkan konsep pemrograman untuk pengembangan perangkat lunak, dalam bentuk UI yang terstruktur dengan layout yang tepat.

Bagian 1

Cara berpikir deklaratif

Kita mendeskripsikan seperti apa layar seharusnya, bukan cara mengubahnya.

Imperatif dan deklaratif

Imperatif, Android View lama

```
val judul = findViewById<TextView>(R.id.judul)
judul.text = laporan.judul
judul.visibility = View.VISIBLE

// status berubah? ubah lagi satu per satu
judul.text = laporanBaru.judul
```

Deklaratif, Compose

```
@Composable
fun Judul(laporan: Laporan) {
    Text(text = laporan.judul)
}

// data berubah? Compose memanggil ulang Judul
```

Analoginya

Di warung, kita cukup bilang pecel lele satu, sambal dipisah. Kita tidak mendikte cara menggoreng lelenya. Compose bekerja seperti itu: kita menyebut hasil yang kita mau, Compose yang mengurus cara menggambarinya.

Aturan main composable function

commonMain/ui/Sapaan.kt

```
@Composable
fun SapaanPelapor(nama: String) {
    Text(
        text = "Halo, $nama",
        fontSize = 24.sp
    )
}

@Composable
fun App() {
    SapaanPelapor(nama = "Rina")
}
```

Aturan	Alasannya
Diberi anotasi @Composable	Compose perlu tahu fungsi mana yang menggambar UI
Dipanggil dari composable lain	Hanya di sana Compose bisa melacak perubahannya
Tidak mengembalikan View	Fungsi ini memancarkan UI, bukan membuat objek
Nama memakai PascalCase	Supaya terbaca sebagai komponen, seperti Text dan Button

Bagian 2

Menyusun layar dengan Column, Row, dan Box

Tiga wadah dasar, ditambah Modifier untuk mengatur ukuran, jarak, warna, dan klik.

Tiga wadah dasar

Column



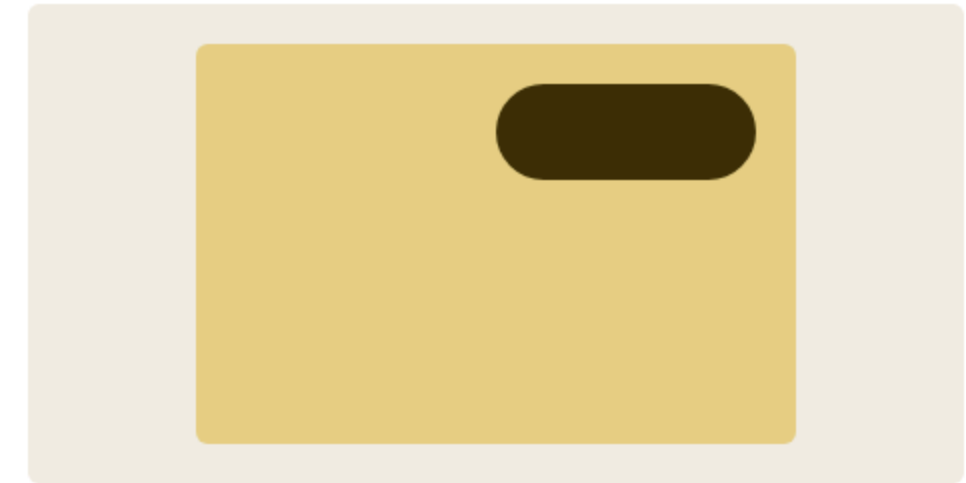
Menyusun anak dari atas ke bawah.
Dipakai untuk layar daftar.

Row



Menyusun anak dari kiri ke kanan.
Dipakai untuk isi satu kartu.

Box



Menumpuk anak di tempat yang sama. Dipakai untuk label di atas foto.

Column untuk layar daftar

commonMain/ui/LayarDaftar.kt

```
@Composable
fun LayarDaftar(semua: List<Laporan>) {
    Column(
        modifier = Modifier.fillMaxSize().padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        Text("LaporKampus", fontSize = 28.sp)
        Text("${semua.size} laporan aktif", color = Color.Gray)
        semua.take(3).forEach { KartuLaporan(it) }
    }
}
```

Parameter	Pilihan
verticalArrangement	Top, Center, SpaceBetween, spacedBy
horizontalAlignment	Start, CenterHorizontally, End

Untuk sementara kita ambil tiga laporan saja. Kalau ada ratusan, Column bukan pilihan yang tepat, dan alasannya kita bahas di bagian 3.

Row untuk isi satu baris

commonMain/ui/BarisLokasi.kt

```
@Composable
fun BarisLokasi(lokalasi: String, waktu: String) {
    Row(
        modifier = Modifier.fillMaxWidth(),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Icon(Icons.Default.Place, contentDescription = null)
        Text(lokalasi, modifier = Modifier.weight(1f))
        Text(waktu, color = Color.Gray)
    }
}
```

`Modifier.weight(1f)` membuat teks lokasi mengambil semua sisa ruang, sehingga waktu terdorong ke ujung kanan.

Tanpa `weight`, ketiganya menempel di kiri dan sisa ruang kosong di kanan.

Box untuk menumpuk label di atas foto

commonMain/ui/FotoLaporan.kt

```
@Composable
fun FotoLaporan(laporan: Laporan) {
    Box(
        modifier = Modifier.fillMaxWidth().height(160.dp),
        contentAlignment = Alignment.TopEnd
    ) {
        FotoAtauKosong(laporan.fotoUrl)
        LabelStatus(laporan.status)
    }
}
```

Anak yang ditulis belakangan digambar paling atas. Foto dulu, baru label status.

`fotoUrl` boleh kosong, ingat tipe nullable minggu lalu. `FotoAtauKosong` menampilkan kotak abu-abu kalau fotonya tidak ada.

Modifier dirangkai, dan urutannya berarti

commonMain

```
Text(  
    text = "Diproses",  
    modifier = Modifier  
        .padding(16.dp)  
        .background(Color(0xFFF6EDCF))  
        .padding(8.dp)  
        .clickable { bukaDetail() }  
)
```

Modifier dibaca dari atas ke bawah, seperti membungkus kado lapis demi lapis.

Baris	Artinya
padding(16.dp)	Jarak di luar warna
background	Warna latar
padding(8.dp)	Jarak di dalam warna
clickable	Area yang bisa diketuk

Tebak dulu sebelum dijalankan

A

Modifier

```
.padding(16.dp)
```

```
.background(Kuning)
```

B

Modifier

```
.background(Kuning)
```

```
.padding(16.dp)
```

TEBAK 1 MENIT

Kedua kode memakai modifier yang sama dengan urutan berbeda. Di mana warna kuning muncul pada A, dan di mana pada B?

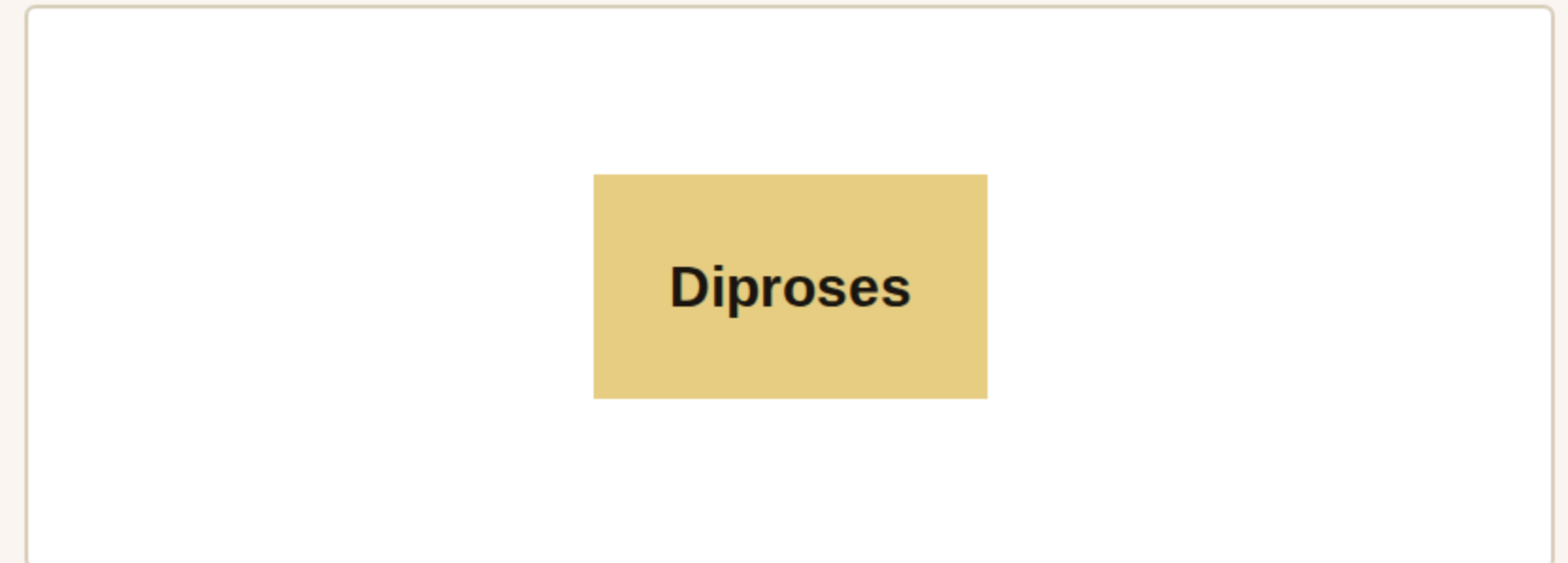
Jawabannya

A padding lalu background



Jarak 16dp tetap transparan. Warna hanya menempel di teks.
Garis putus-putus menandai jaraknya.

B background lalu padding



Warna dipasang dulu, lalu jarak ditambahkan di dalamnya, jadi kuning ikut melebar.

Modifier yang paling sering dipakai

Kelompok	Contoh	Di Laporkampus
Ukuran	<code>fillMaxWidth()</code> , <code>size(48.dp)</code> , <code>height(160.dp)</code>	Foto laporan selebar layar
Jarak	<code>padding(16.dp)</code> , <code>padding(horizontal = 8.dp)</code>	Ruang di dalam kartu
Latar	<code>background(warna, RoundedCornerShape(50))</code>	Label status berbentuk pil
Bingkai	<code>border(1.dp, Color.Gray)</code> , <code>clip(CircleShape)</code>	Foto profil pelapor bulat
Interaksi	<code>clickable { }</code>	Kartu bisa diketuk untuk membuka detail

Bagian 3

Komponen dan composable yang bisa dipakai ulang

Text, Button, Image, Card, lambda dan koleksi seperlunya, lalu daftar panjang dengan LazyColumn.

Text dan tombol

Komponen	Dipakai untuk
Text	Judul, lokasi, keterangan
Button	Aksi utama, misalnya Kirim laporan
OutlinedButton	Aksi kedua, misalnya Simpan draf
TextButton	Aksi ringan, misalnya Batal
FloatingActionButton	Satu aksi terpenting di layar

commonMain

```
FloatingActionButton(onClick = { bukaForm() }) {  
    Icon(  
        Icons.Default.Add,  
        contentDescription = "Buat laporan"  
    )  
}  
  
Text(  
    text = laporan.judul,  
    fontWeight = FontWeight.Bold,  
    maxLines = 1,  
    overflow = TextOverflow.Ellipsis  
)
```

commonMain

```
Image(  
    painter = painterResource(Res.drawable.logo_kampus),  
    contentDescription = "Logo kampus",  
    modifier = Modifier.size(48.dp).clip(CircleShape)  
)  
  
Icon(  
    imageVector = Icons.Default.Warning,  
    contentDescription = "Prioritas tinggi",  
    tint = Color(0xFF8C1515)  
)
```

`contentDescription` dibacakan pembaca layar untuk pengguna tunanetra. Isi dengan arti gambarnya, atau null kalau hanya hiasan.

Foto laporan dari internet butuh library tambahan seperti Coil. Kita pasang saat data mulai datang dari server di minggu 6.

KartuLaporan, menyatukan semuanya

commonMain/ui/KartuLaporan.kt

```
@Composable
fun KartuLaporan(laporan: Laporan) {
    Card(elevation = CardDefaults.cardElevation(2.dp)) {
        Row(
            modifier = Modifier.fillMaxWidth().padding(16.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Column(Modifier.weight(1f)) {
                Text(laporan.judul, fontWeight = FontWeight.Bold)
                Text(laporan.lokasi, color = Color.Gray)
            }
            LabelStatus(laporan.status)
        }
    }
}
```

Proyektor mati

GPU 1 R201

Baru

Card berisi Row, Row berisi Column dan label.
Persis seperti hasil diskusi di awal kelas.

Fungsi juga bisa dikirim sebagai nilai

Main.kt

```
fun ulangi(kali: Int, aksi: (Int) → Unit) {  
    for (i in 1..kali) aksi(i)  
}  
  
ulangan(3) { println("Laporan ke-$it") }  
  
val tandaiDibaca: (Laporan) → Unit = {  
    println("#${it.id} sudah dibaca")  
}  
tandaiDibaca(laporan)
```

Penulisan	Artinya
() -> Unit	Fungsi tanpa masukan dan tanpa hasil
(Int) -> Unit	Fungsi dengan satu masukan Int
{ ... }	Lambda, fungsi tanpa nama
it	Nama otomatis untuk satu masukan

Lambda di argumen terakhir boleh ditulis di luar kurung, seperti `ulangan(3) { }`.

Parameter membuat komponen bisa dipakai ulang

commonMain/ui/KartuLaporan.kt

```
@Composable
fun KartuLaporan(
    laporan: Laporan,           // wajib diisi
    ringkas: Boolean = false,    // punya nilai bawaan
    catatan: String? = null,     // boleh kosong
    onKlik: () → Unit = {}      // dipanggil saat diketuk
)

KartuLaporan(laporan)
KartuLaporan(laporan, ringkas = true)
KartuLaporan(laporan, onKlik = { buka(laporan.id) })
```

Satu komponen, banyak pemakaian. Layar daftar memakai versi lengkap, notifikasi memakai versi ringkas.

Komponen tidak tahu apa yang terjadi saat diketuk. Ia hanya memanggil **onKlik**, dan pemanggilnya yang memutuskan.

Mengolah daftar sebelum ditampilkan

commonMain/ui/LayarDaftar.kt

```
val baru = semua.filter { it.status is StatusLaporan.Baru }  
val judul = semua.map { it.judul }  
val terbaru = semua.sortedByDescending { it.id }  
val perLokasi = semua.groupBy { it.lokasi }  
val adaFoto = semua.count { it.fotoUrl != null }
```

```
DaftarLaporan(  
    semua = terbaru.take(20),  
    onPilih = { bukaDetail(it.id) }  
)
```

Fungsi	Hasilnya
filter	Daftar yang lolos syarat
map	Daftar baru hasil ubahan
sortedByDescending	Daftar terurut menurun
groupBy	Peta kelompok per kunci
count	Jumlah yang lolos syarat

Daftar panjang dengan LazyColumn

commonMain/ui/DaftarLaporan.kt

```
@Composable
fun DaftarLaporan(
    semua: List<Laporan>,
    onPilih: (Laporan) → Unit
) {
    LazyColumn(
        contentPadding = PaddingValues(16.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        items(semaua, key = { it.id }) { laporan →
            KartuLaporan(laporan, onKlik = { onPilih(laporan) })
        }
    }
}
```

LazyColumn hanya menggambar kartu yang sedang terlihat di layar, lalu memakai ulang tempatnya saat pengguna menggulir.

key membantu Compose mengenali laporan yang sama walaupun urutannya berubah.

BANGUN BARENG

Column dengan forEach, atau LazyColumn?

LaporKampus punya 500 laporan. Kalau semuanya ditampilkan dengan Column dan forEach, apa yang terjadi saat layar pertama kali dibuka? Kapan Column dengan forEach masih pantas dipakai?

Komponen lain yang akan sering kita temui

Komponen	Gunanya	Contoh di LaporKampus
TextField	Masukan teks	Judul laporan, dibahas lengkap minggu 4
Checkbox	Pilihan ya atau tidak	Tandai laporan sudah dibaca
Switch	Hidup atau mati	Notifikasi status laporan
RadioButton	Satu dari beberapa pilihan	Kategori kerusakan
Slider	Nilai dalam rentang	Tingkat keparahan 1 sampai 5
CircularProgressIndicator	Sedang memuat	Menunggu daftar dari server

Bagian 4

Coba sendiri, lalu bawa ke proyekmu

Latihan Compose dan koleksi dengan observasi praktik, layar daftar tema, dan checkpoint proyek.

Yang saya amati selama latihan hari ini

Indikator RPS: **UI terstruktur dengan layout yang tepat.** Diamati dari latihan 1 dan 3, ProfileCard dan ProductList.

Kriteria	Sangat baik 85 sampai 100	Baik 70 sampai 84	Cukup 55 sampai 69	Kurang di bawah 55
UI terstruktur CPMK0501, 4%	Kartu profil dan daftar produk rapi, urutan modifier tepat, daftar memakai LazyColumn	Keduanya tampil, ada jarak atau perataan yang kurang tepat	Hanya satu yang tampil, layout berantakan	Belum ada tampilan yang jalan

Individu, bobot 4% nilai akhir. Bukti: push ke repository latihan pribadimu, lalu kirim tautan commit di LMS sebelum kelas selesai.

COBA SENDIRI

Tiga latihan di repo mata kuliah



Compose, latihan 1

Handson1ProfileCard.kt

Avatar bulat, nama tebal, dan bio abu-abu di dalam Card.

Compose, latihan 3

Handson3ProductList.kt

Kartu berisi kotak warna, nama, dan harga, diulang untuk setiap produk.

Kotlin Dasar P4, latihan 1

handson1-latihan

Nama produk di atas harga tertentu, terurut dari yang termurah, dengan filter, sortedBy, dan map.

Folder **P3 Compose Multiplatform Basics Hands-on** dan **Kotlin Dasar, P4 Collections and co. Hands-on**. Latihan 2 Compose (form login) boleh dikerjakan kalau waktu tersisa.

Kalau error

Pesan error	Penyebab	Yang dicoba
@Composable invocations can only happen from the context of a @Composable function	Memanggil composable dari fungsi biasa	Beri anotasi @Composable pada pemanggilnya
Unresolved reference: dp atau sp	Import belum ada	import androidx.compose.ui.unit.dp
Type mismatch: expected Color	Color dari package yang salah	Pakai androidx.compose.ui.graphics.Color
Unresolved reference: Icons	Import ikon belum lengkap	Tambahkan import Icons dan ikon yang dipakai

Layar daftar untuk tema kelompokmu

1 **Kartu entitas**
Composable kartu untuk entitas utama tema kalian, dengan label status.

2 **Daftar dummy**
LazyColumn berisi minimal sepuluh data contoh dari data class minggu lalu.

3 **Satu aksi utama**
Tombol atau FAB untuk menambah data, belum perlu berfungsi.

4 **Push ke repository**
Tangkapan layar di README kelompok.

CONTOH DARI LAPORKAMPUS

KartuLaporan untuk entitas Laporan, LabelStatus untuk StatusLaporan, dan FAB Lapor di pojok kanan bawah.

Checkpoint proyek minggu 3

- 1** **Layar daftar**
LazyColumn berisi kartu entitas utama dan label status dari sealed class minggu lalu.
- 2** **Olah sebelum tampil**
Minimal satu filter atau urutan dengan fungsi koleksi, misalnya terbaru di atas.
- 3** **Satu aksi utama**
FAB tambah data, boleh belum berfungsi, dan tangkapan layar di README.

Dikumpulkan

Push ke repository kelompok. README memuat tangkapan layar Android atau desktop dan pembagian kerja.

Dicek di awal pertemuan berikutnya: tuntas, sebagian, atau belum.

TUTUP

Tiket keluar



1

Deklaratif

Apa yang terjadi pada tampilan Compose saat data yang dipakainya berubah?

2

Modifier

Kenapa padding sebelum background berbeda hasilnya dengan padding sesudah background?

3

Daftar

Kapan kalian memilih LazyColumn, dan kapan Column dengan forEach sudah cukup?

Jawab singkat di kertas atau formulir LMS sebelum keluar kelas.

TUTUP

Minggu depan: state dan MVVM



MASALAH MINGGU 4

Kita membuat form laporan baru. Saat mahasiswa mengetik judul, kenapa teks di layar tidak ikut berubah? Dan kenapa isi form hilang saat layar diputar?

Persiapan

Bawa layar daftar kelompok kalian. Mulai minggu depan tidak ada lagi Kotlin seperlunya, kita fokus ke state dan arsitektur.

Terima kasih

Sampai jumpa di pertemuan 4

Ada pertanyaan? Sampaikan sekarang, lewat LMS, atau email ke muhammad.algifari@if.itera.ac.id